Application of Kolmogorov's Superposition Theorem to Regression Problems

Yaroslav Lyulko

International College of Economics and Finance (ICEF)

AIMF Lab, HSE

October 13, 2026

Overview

- Motivation
- Theoretical Considerations
- Proposed Algorithm
- Conclusions
- References

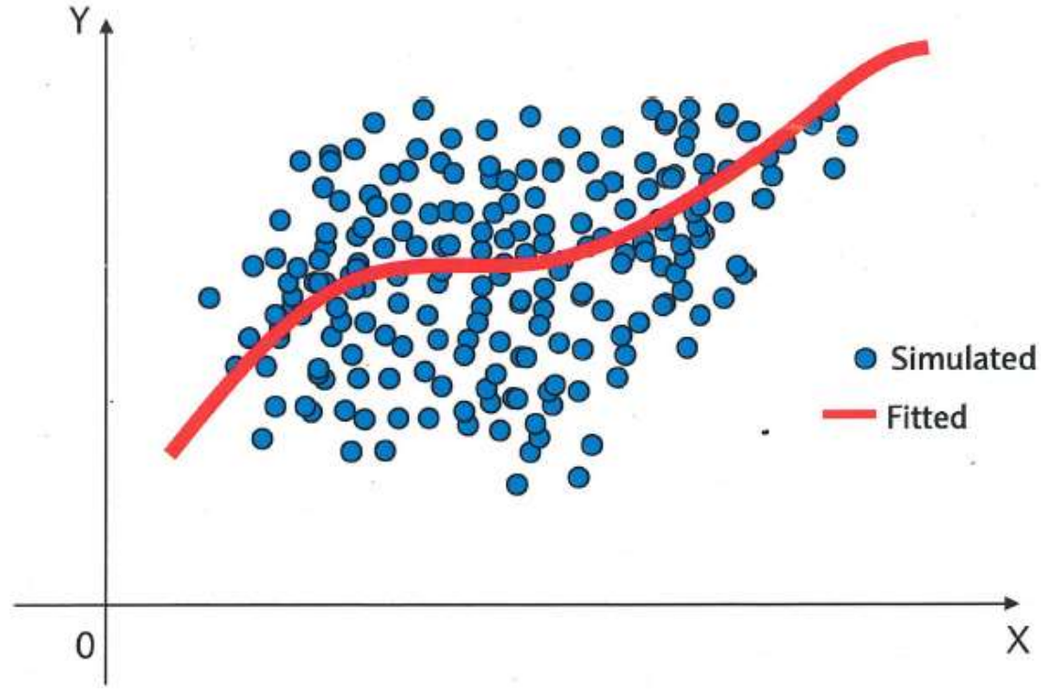
Regression in Quantitative analytics

We use regression in Quantitative analytics for various purposes

Use – cases:

- Longstaff-Schwarz (LS) Framework
- Monte Carlo Unified Trade Representation
- Index strategy replication

……



Regression in Quantitative analytics

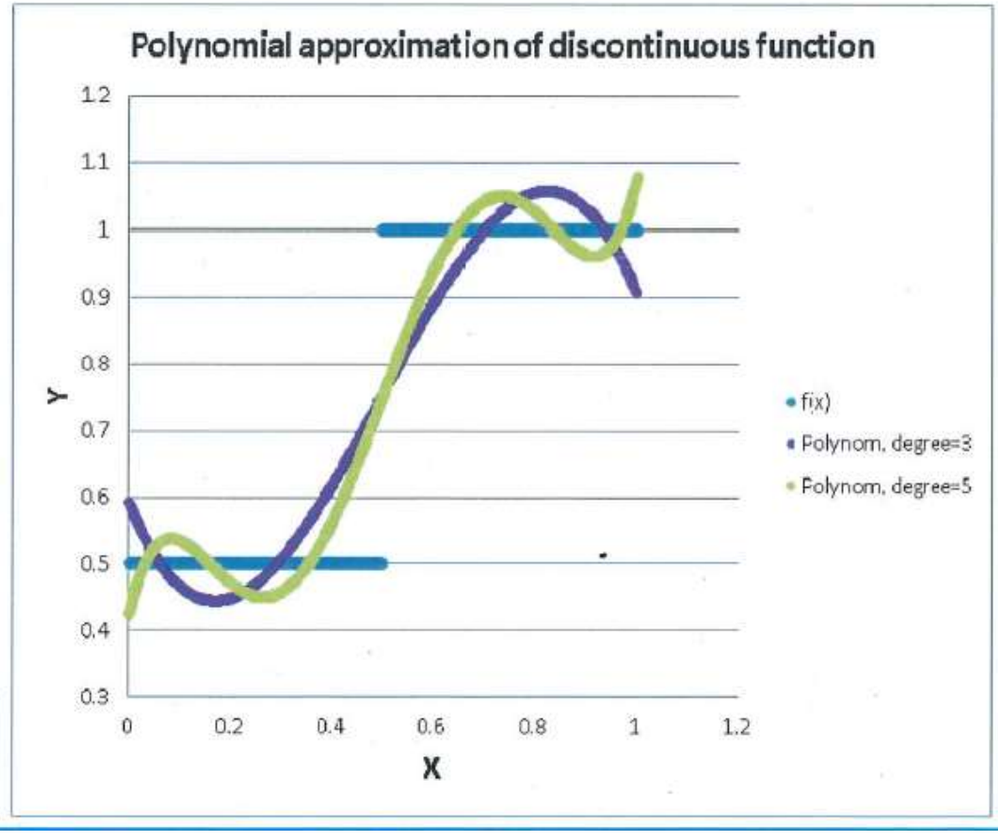
Most regression methods implemented in QA are currently **polynomial-based**

Weaknesses of polynomial algorithms

- weak approximation power for non-smooth functions especially for **discontinuous** ones;
- approximation power of polynomial regression significantly depends on **degree which is a user-defined parameter**
- «**curse of dimensionality**»: exponential increase in the volume of calculations when adding independent variables

Our task is to develop a new regression method which is not based on polynomials

$$y(x) = 0.5 \cdot \mathbb{I}_{[0;\,0.5)}(x) + \mathbb{I}_{[0.5;\,1]}(x)$$



TARN instrument (FX)

Example in 2D (d=2) : TARN trade

- Let FX_1 = AUDJPY, FX_2 = USDJPY, spread = $FX_1 - FX_2$
- Let $T_1 < T_2 < \ldots < T_n$ be n **fixing dates**, $T > T_n$ – **valuation date**
- $A(T_i) = \begin{cases} 1 & \text{if } \text{spread}(T_i) > B \\ 0 & \text{if } \text{spread}(T_i) \le B \end{cases}$ – accrued amount at fixing date T_i, B – **spread boundary**

$$A(t) = \sum_{i:\, T_i \le t} A(T_i) \quad \text{– accrued amount until date t}$$

- **Payoff** (T) = $\sum_{i=1}^{n} \text{Notinal}_i \cdot (FX_1(T_i) - FX_2(T_i) - K)$. *We have a **Basket Forward Strip***

which pays coupons over time until the accrued amount A(·) reaches a given TARN level

x_1 = accrued amount,
x_2 = spread = $FX_1 - FX_2$

y – value of the trade close to expiry

Theoretical Considerations

Kolmogorov's Superposition Theorem (KST)

Theorem [Kolmogorov, 1957]

For each integer $d \geq 2$ there exist continuous functions $\varphi_q: [0,1] \to \mathbb{R}$, $q=0 \dots 2d$ and constants $\lambda_1, \dots, \lambda_d \in \mathbb{R}$ such that for every **continuous multivariate function** $y : [0,1]^d \to \mathbb{R}$ there exist continuous univariate function $g : [0,1] \to \mathbb{R}$ such that the following representation holds:

$$y(x_1, \dots, x_d) = \sum_{q=0}^{2d} g\left[\lambda_1 \varphi_q(x_1) + \lambda_2 \varphi_q(x_2) + \dots + \lambda_d \varphi_q(x_d) \right]$$

Kolmogorov's Superposition Theorem (KST)

Theorem [Kolmogorov, 1957]

For each integer $d \geq 2$ there exist continuous functions $\varphi_q: [0,1] \to \mathbb{R}$, $q=0 \dots 2d$ and constants $\lambda_1, \dots, \lambda_d \in \mathbb{R}$ such that for every **continuous multivariate function** $y : [0,1]^d \to \mathbb{R}$ there exist continuous univariate function $g : [0,1] \to \mathbb{R}$ such that the following representation holds:

$$y(x_1, \dots, x_d) = \sum_{q=0}^{2d} g\left[\lambda_1 \varphi_q(x_1) + \lambda_2 \varphi_q(x_2) + \dots + \lambda_d \varphi_q(x_d) \right]$$

function $g(\cdot)$ depends on $y(x_1, \dots, x_d)$

these terms do not depend on $y(x_1, \dots, x_d)$

Kolmogorov-Sprecher approach

In 1996 D. Sprecher proved a **computable version** of Kolmogorov's Superposition theorem which became a starting point for applications

For any regression problem we specify **input points** $\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_d, \vec{y}$:

$(\vec{x}^1, y^1)$

$$\vec{X}_1 = \begin{pmatrix} X_1^1 \\ X_1^2 \\ \cdot \\ \cdot \\ \cdot \\ X_1^N \end{pmatrix} \quad \vec{X}_2 = \begin{pmatrix} X_2^1 \\ X_2^2 \\ \cdot \\ \cdot \\ \cdot \\ X_2^N \end{pmatrix} \quad \ldots\ldots \quad \vec{X}_d = \begin{pmatrix} X_d^1 \\ X_d^2 \\ \cdot \\ \cdot \\ \cdot \\ X_d^N \end{pmatrix} \quad \vec{y} = \begin{pmatrix} y^1 \\ y^2 \\ \cdot \\ \cdot \\ \cdot \\ y^N \end{pmatrix}$$

Kolmogorov-Sprecher approach

Apply a change of variables

$(x_1, x_2, \ldots, x_d) \mapsto (u_0, u_1, \ldots, u_d, u_{d+1}, \ldots, u_{2d})$ defined by

$$\begin{cases} u_0(\vec{x^i}) = \lambda_1 \varphi_0(x_1^i) + \lambda_2 \varphi_0(x_2^i) + \ldots + \lambda_d \varphi_0(x_d^i) \\ u_1(\vec{x^i}) = \lambda_1 \varphi_1(x_1^i) + \lambda_2 \varphi_1(x_2^i) + \ldots + \lambda_d \varphi_1(x_d^i) \\ \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \\ u_{2d}(\vec{x^i}) = \lambda_1 \varphi_{2d}(x_1^i) + \lambda_2 \varphi_{2d}(x_2^i) + \ldots + \lambda_d \varphi_{2d}(x_d^i) \end{cases}$$

With this change of variables we identify

$$y(x_1, \ldots, x_d) = \sum_{q=0}^{2d} g\Big(\underbrace{\lambda_1 \varphi_q(x_1) + \lambda_2 \varphi_q(x_2) + \ldots + \lambda_d \varphi_q(x_d)}_{u_q(\vec{x})} \Big)$$

Kolmogorov-Sprecher approach

The computable version of the superposition theorem provides us with **explicit expressions** for $\lambda_1, \lambda_2, \ldots, \lambda_d$ and for functions $\varphi_0(x), \varphi_1(x), \ldots, \varphi_{2d}(x)$

- $\lambda_1 = 0.5$; $\lambda_{i+1} = \frac{1}{2}\sum_{j=1}^{+\infty}(2d+2)^{-i\frac{d^j-1}{d-1}}$, $i \geq 1$

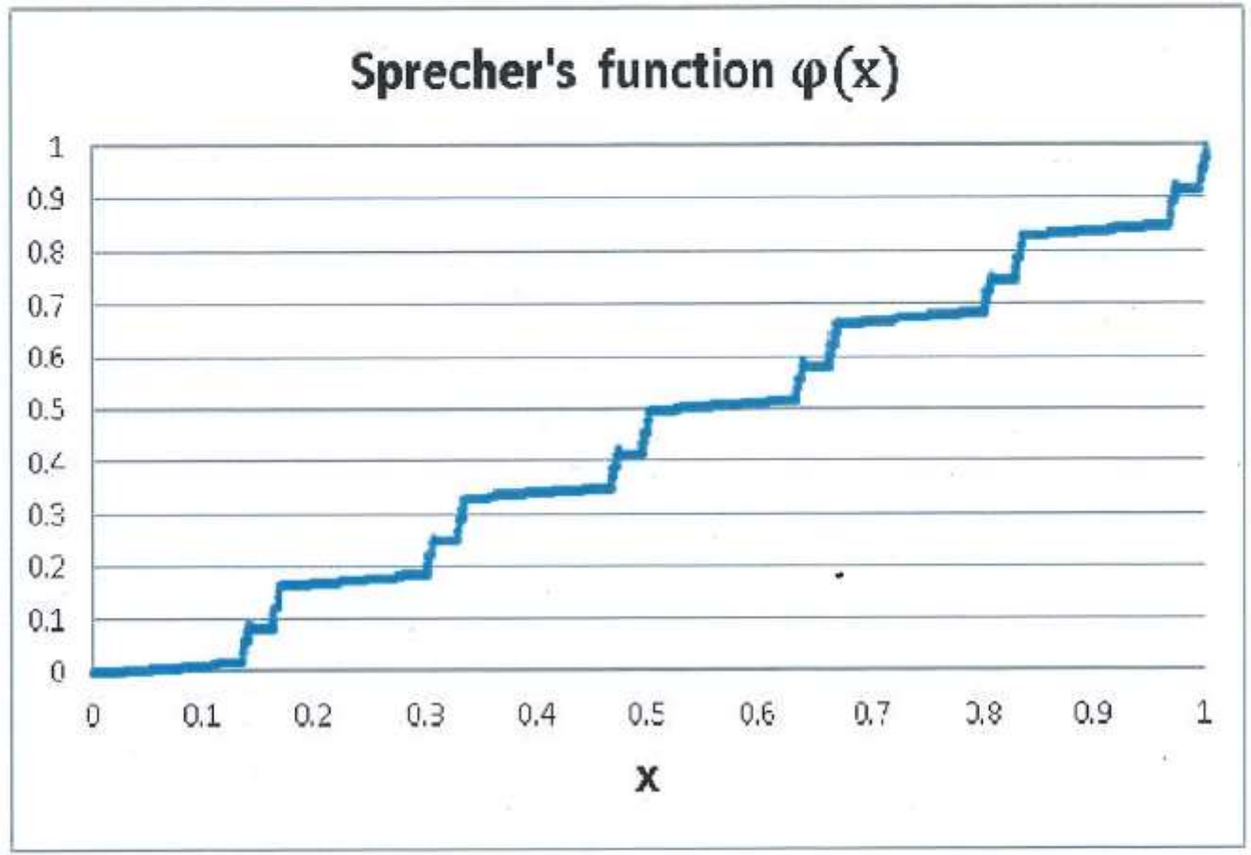
  $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_d$

- $\varphi_q(x) = a_q\, \varphi(x + b_q) + c_q$

  $\varphi(x)$ – **Sprecher's function**

  - $\varphi : [0, 1] \rightarrow [0, 1]$
  - extension

    $\varphi(x + 1) = \varphi(x) + 1,\ x \in [0, 1]$
  - continuous



Kolmogorov-Sprecher approach

- Example in 2D (d=2):

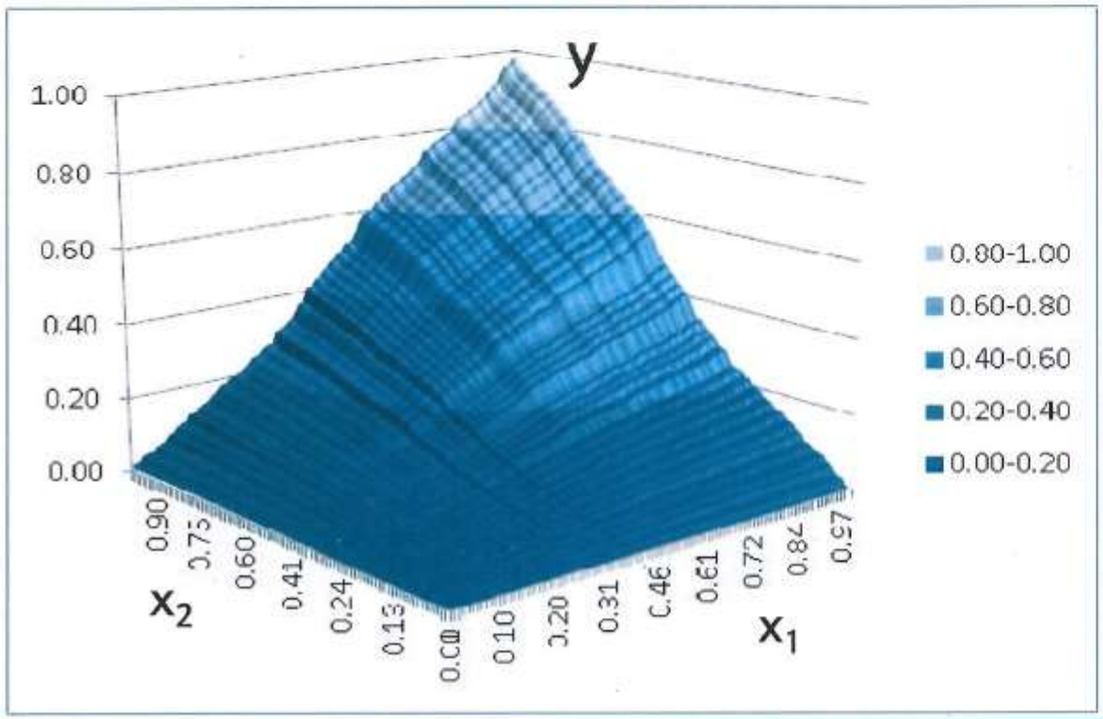
$$y(x_1, x_2) = x_1 \cdot x_2, \quad N = 10000$$

- Sprecher's approach yields $(\hat{y}^i)_{i=1}^N$ – approximation of input points $(y^i)_{i=1}^N$ in the form

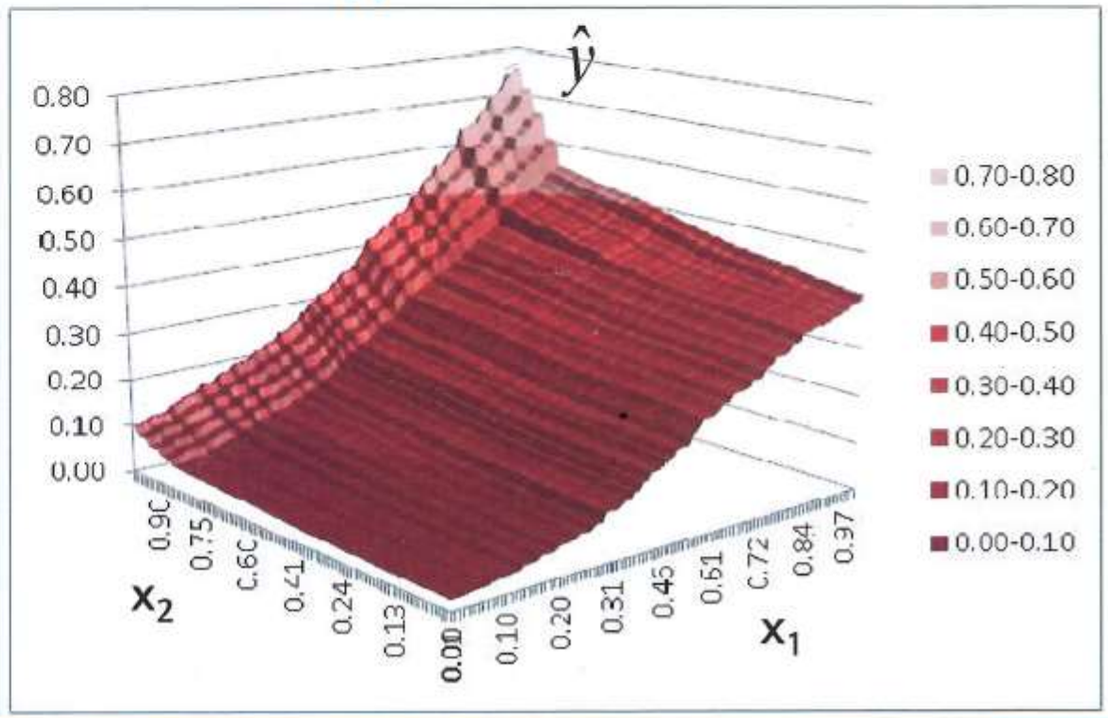
$$\hat{y}^i = g(u_0(\vec{x^i})) + g(u_1(\vec{x^i})) + \ldots\ldots + g(u_{2d}(\vec{x^i})), \quad i = 1 \ldots N$$

- We solve this problem by Additive regression method

Initial surface



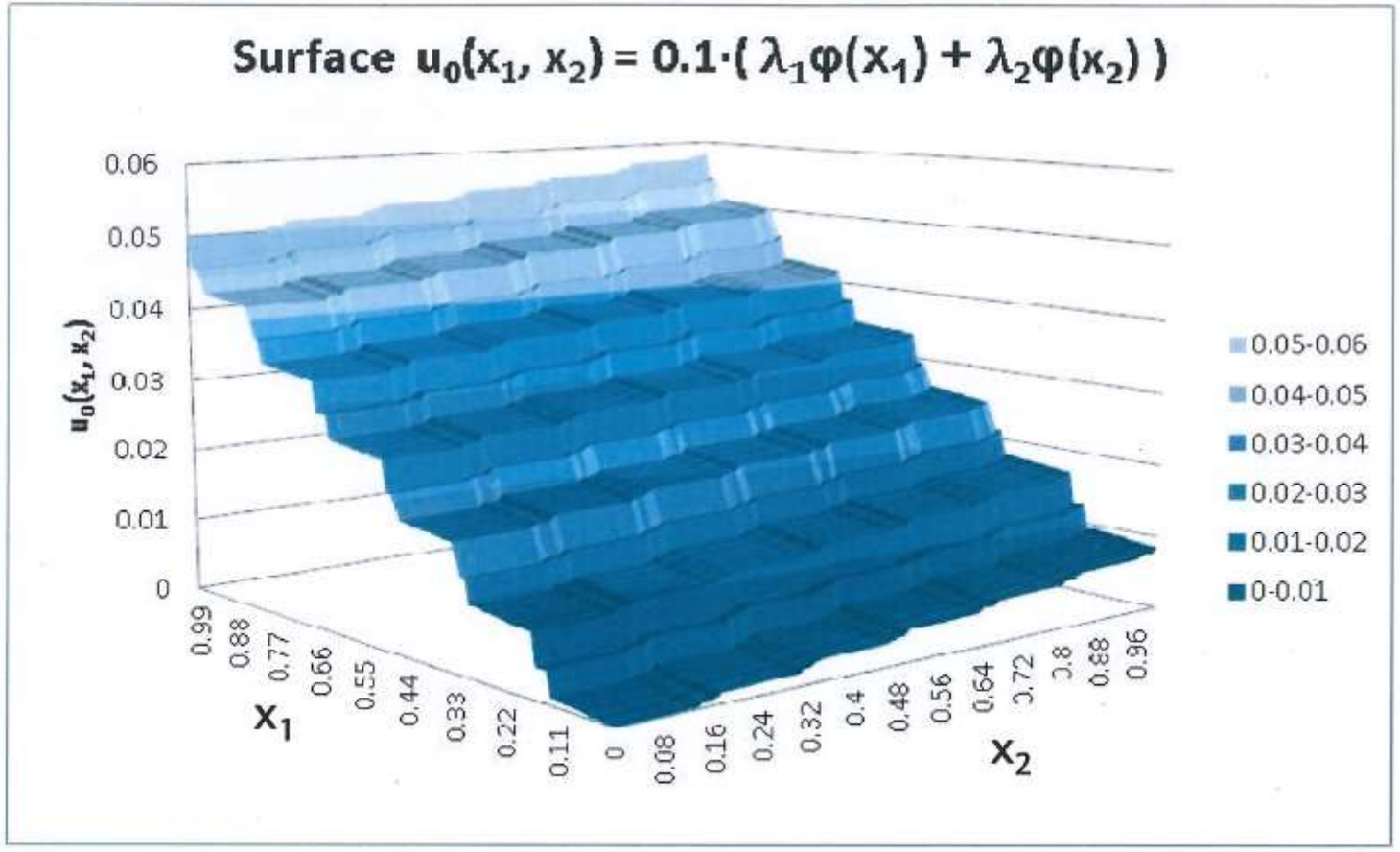
Approximated surface



Kolmogorov-Sprecher approach

Results are clearly not good enough. Why?

- Sprecher's function $\varphi(x)$ is highly non-smooth
- The contribution of each variable $x_1, \ldots, x_d$ to new variables $u_q(\vec{x})$, q=0,…,2d is different



$\lambda_1, \ldots, \lambda_d$, d = 5

i	λ_i
1	0. 5
2	0. 04167
3	0. 00347
4	0. 00029
5	0. 00002

Kolmogorov-Sprecher approach

How can we improve Sprecher's algorithm results?

- Turn down the idea of precomputing any variables $\lambda_1, \lambda_2, \ldots, \lambda_d$ and functions $\varphi_0(x), \varphi_1(x), \ldots, \varphi_{2d}(x)$

- Do not use a fixed number of terms (i.e. 2d) in representation

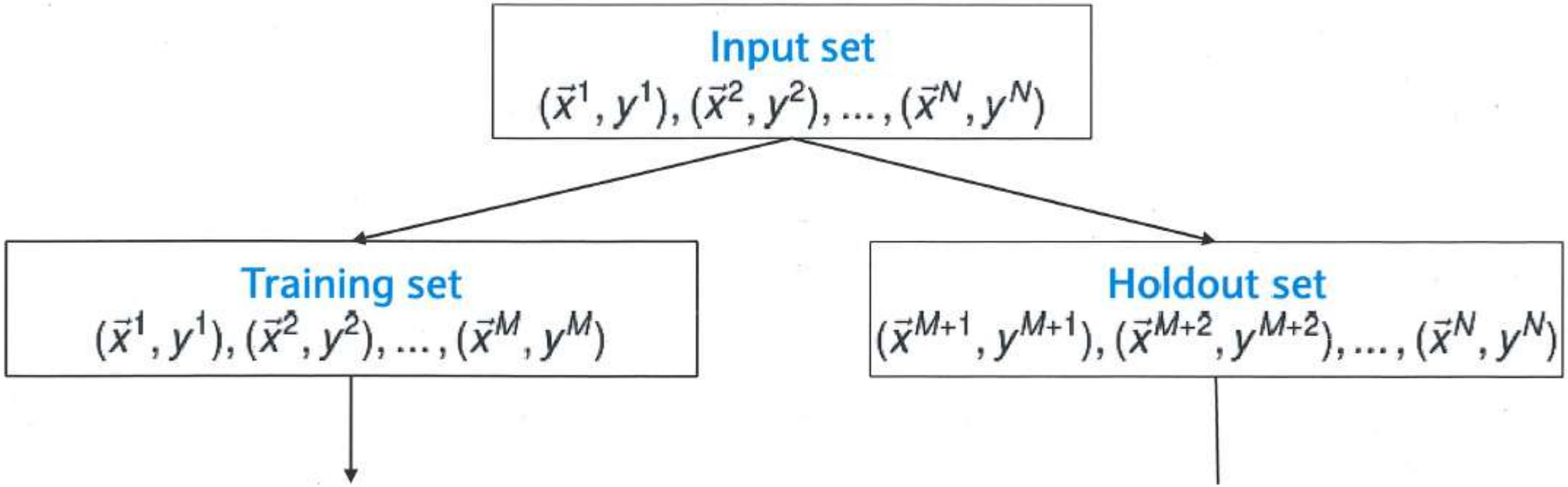
$$y(x_1, \ldots, x_d) = \sum_{q=0}^{2d} g\left[\lambda_1 \varphi_q(x_1) + \lambda_2 \varphi_q(x_2) + \ldots + \lambda_d \varphi_q(x_d) \right]$$

Apply the general form of Superposition theorem but with functions and parameters dependent on $y(\vec{x})$

Proposed Algorithm

Mean Squared Error (MSE)

Let $\vec{x} = (x_1, x_2, \ldots, x_d)$ be d independent variables, y – dependent variable



1 Algorithm $\hat{y}$: values $\hat{y}(x_1, x_2, \ldots, x_d)$ can be calculated $\forall (x_1, x_2, \ldots, x_d) \in$ Domain

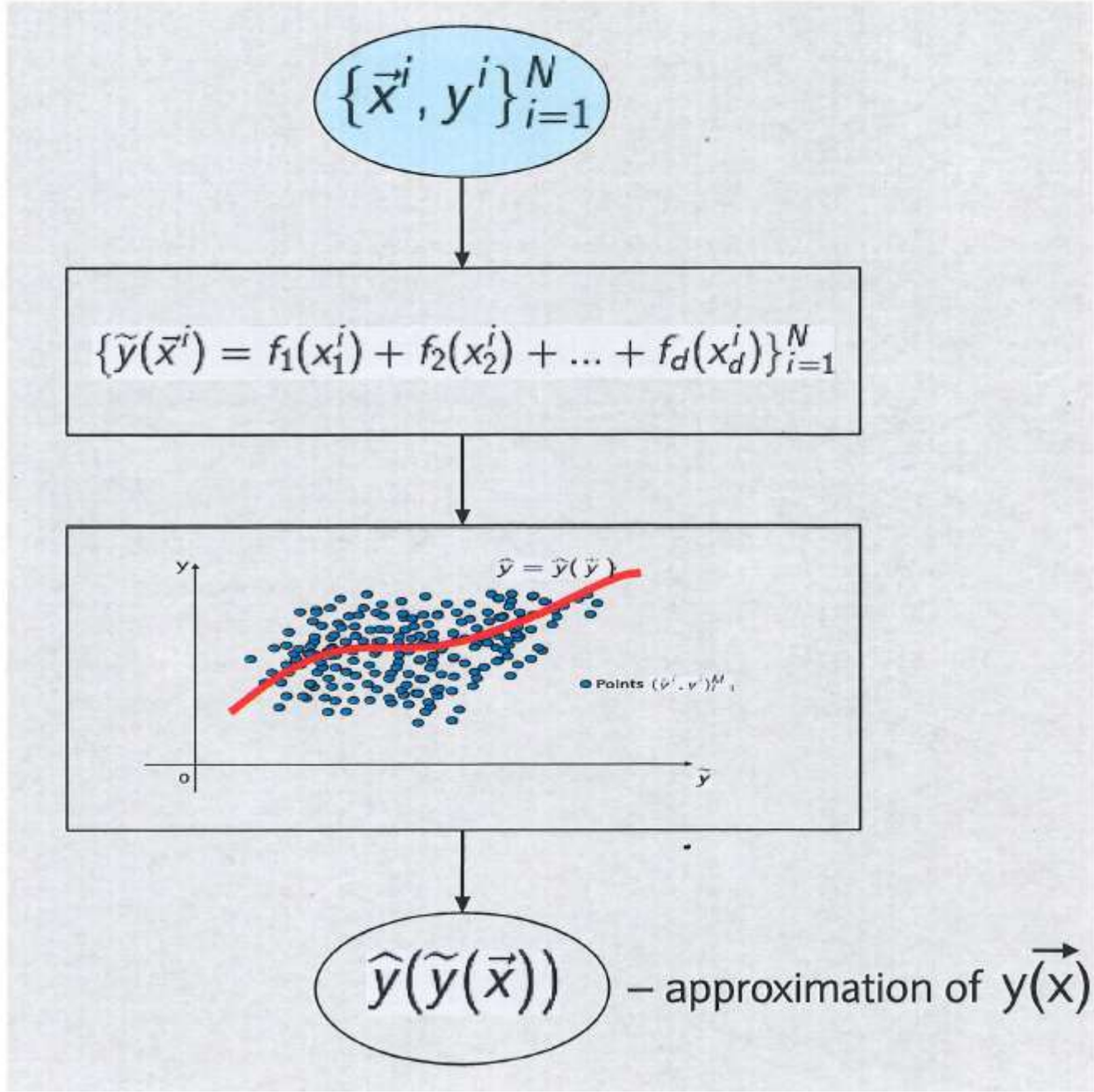
2 calculate a measure of the error of algorithm $\hat{y}$:

$$MSE = \frac{1}{N-M} \sum_{i=M+1}^{N} \left(y^i - \hat{y}(x_1^i, x_2^i, \ldots, x_d^i)\right)^2$$

Description of Kolmogorov-Pednault (KP) algorithm

Let $(x_1^i, x_2^i, \dots, x_d^i, y^i)_{i=1}^N$ be

a set of input points



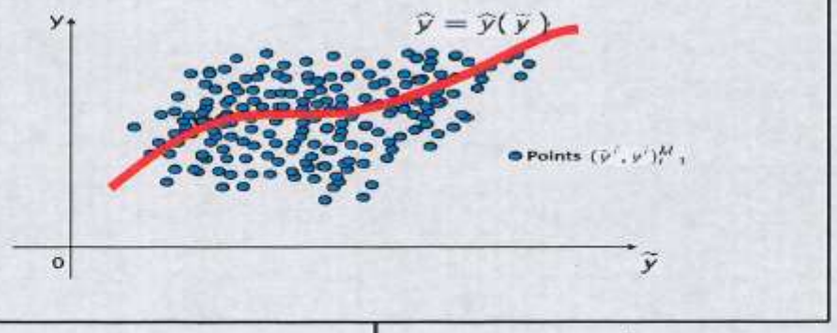
Description of Kolmogorov-Pednault (KP) algorithm

Additive regression

(backfitting algorithm):

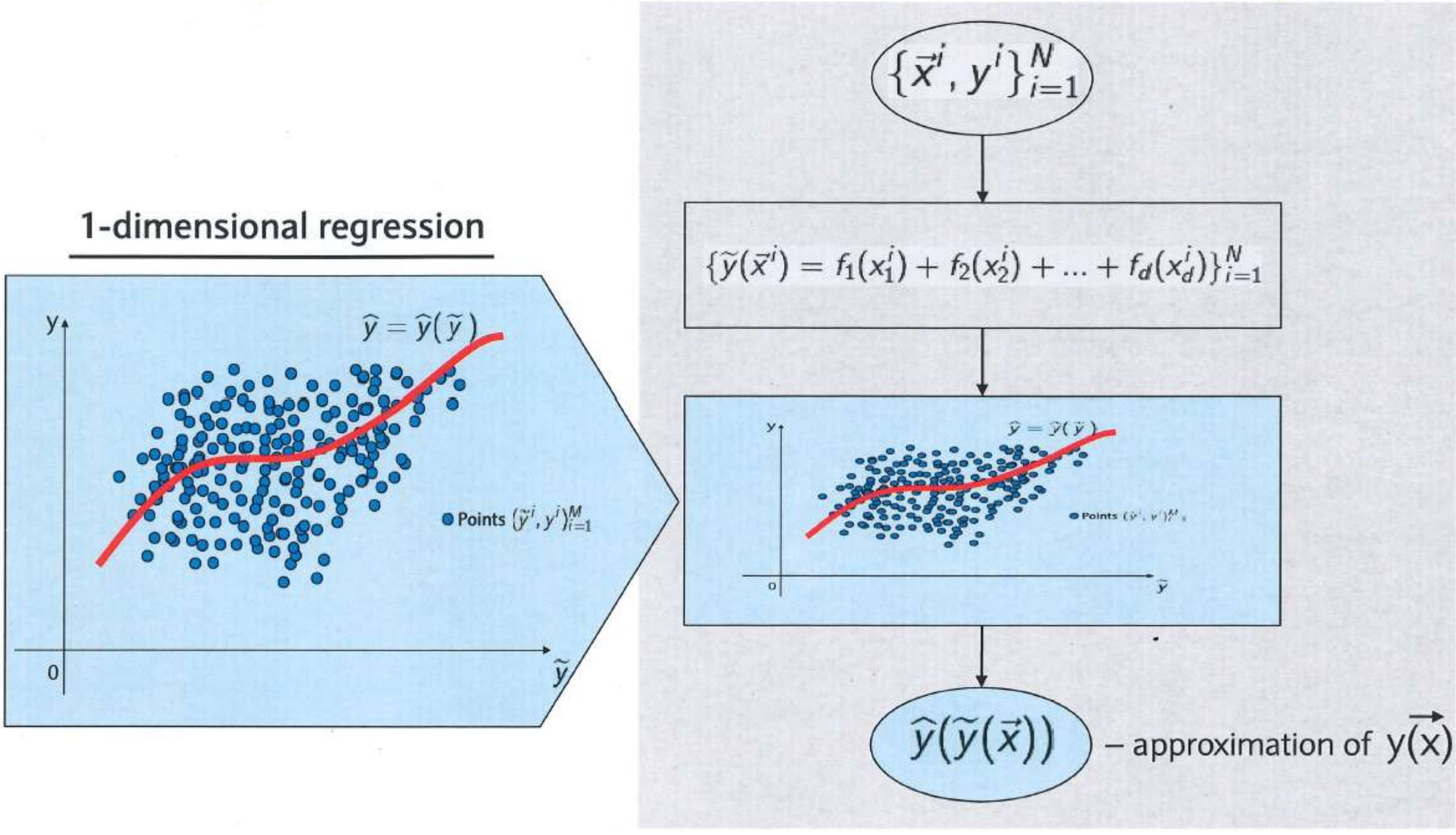
$\{\vec{x}^i, y^i\}_{i=1}^N$

$\{\tilde{y}(\vec{x}^i) = f_1(x_1^i) + f_2(x_2^i) + \ldots + f_d(x_d^i)\}_{i=1}^N$



$\hat{y}(\tilde{y}(\vec{x}))$ – approximation of $y(\vec{x})$

Description of Kolmogorov-Pednault (KP) algorithm



One-dimensional smoother

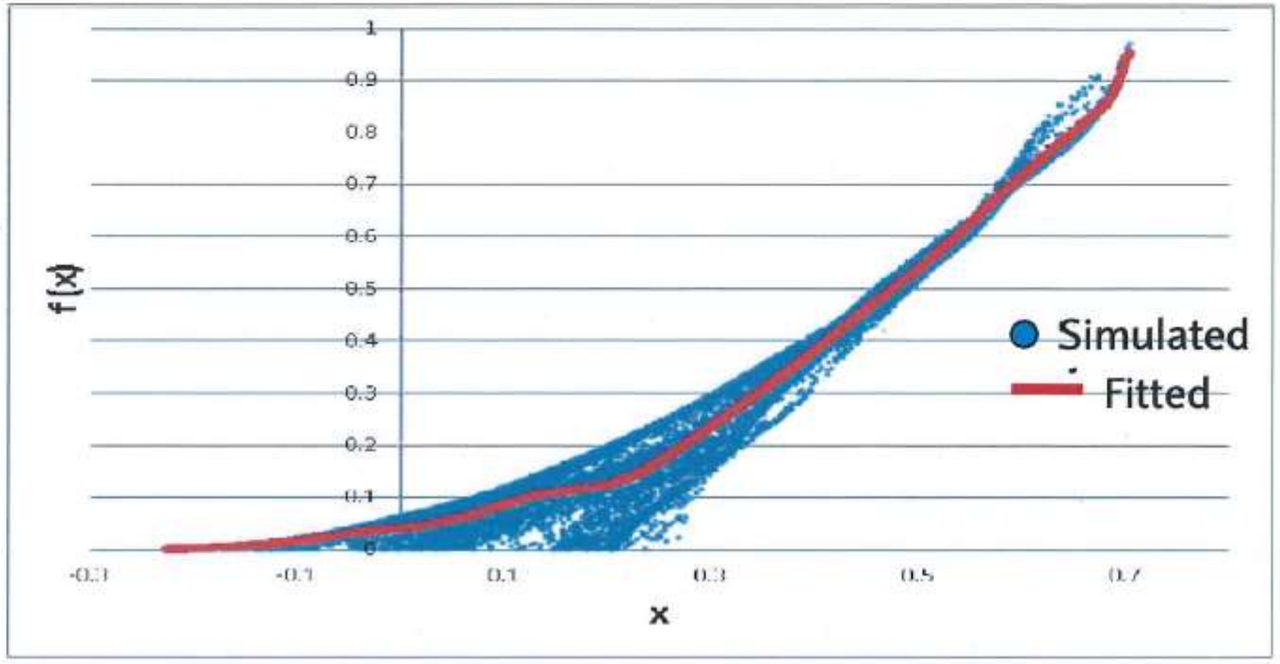
- 1D regression or **smoother** plays a key role in KP algorithm
- We use Nadaraya-Watson kernel-weighted average. Let $(x^1, y^1), \ldots, (x^N, y^N)$ be input points. Then

$$f(x) = \frac{\sum_{i=1}^{N} K\left(\frac{|x - x^i|}{h(x)}\right) y^i}{\sum_{i=1}^{N} K\left(\frac{|x - x^i|}{h(x)}\right)}$$

is our smoother with Gaussian kernel

$$K(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$ and **window width** h = h(x)

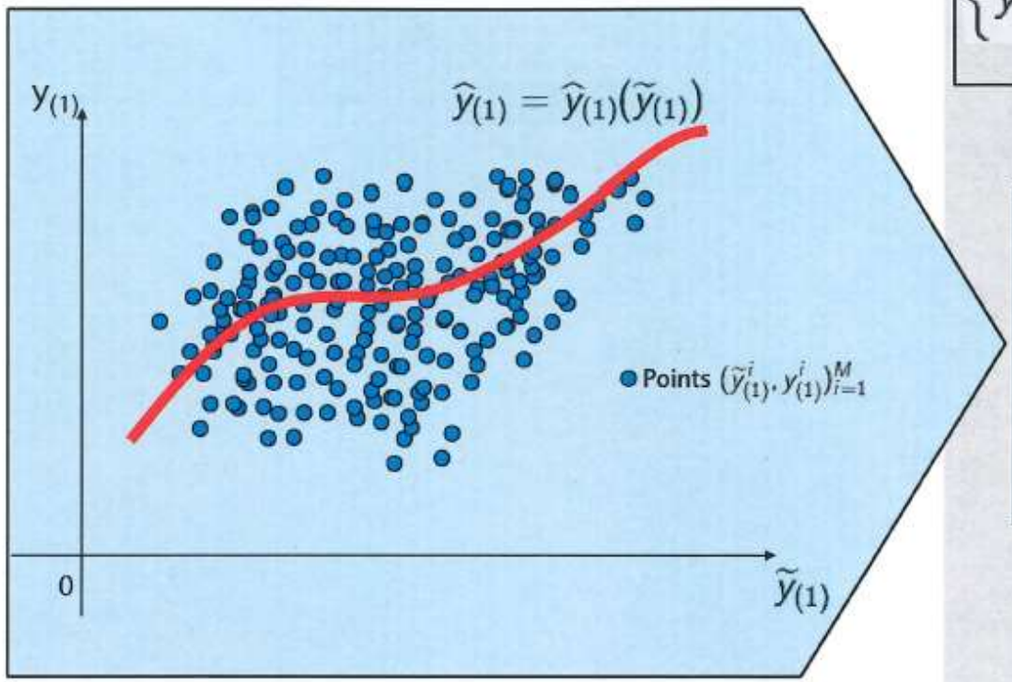
- Typical smoothing:



Description of Kolmogorov-Pednault (KP) algorithm

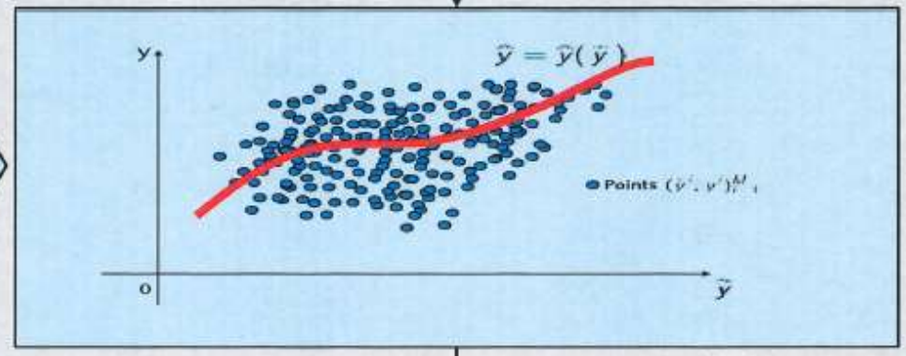
I Boosting stage

1-dimensional regression



$\{\vec{x}^i, y^i_{(1)}\}_{i=1}^N$ $\quad y^i_{(1)} = y^i$

$\left\{\widetilde{y}_{(1)}(\vec{x}^i) = f_1^{(1)}(x_1^i) + f_2^{(1)}(x_2^i) + \ldots + f_d^{(1)}(x_d^i)\right\}_{i=1}^N$



$\widehat{y}_{(1)}(\widetilde{y}_{(1)}(\vec{x}))$ – approximation of $y_{(1)}(\vec{x})$

Description of Kolmogorov-Pednault (KP) algorithm

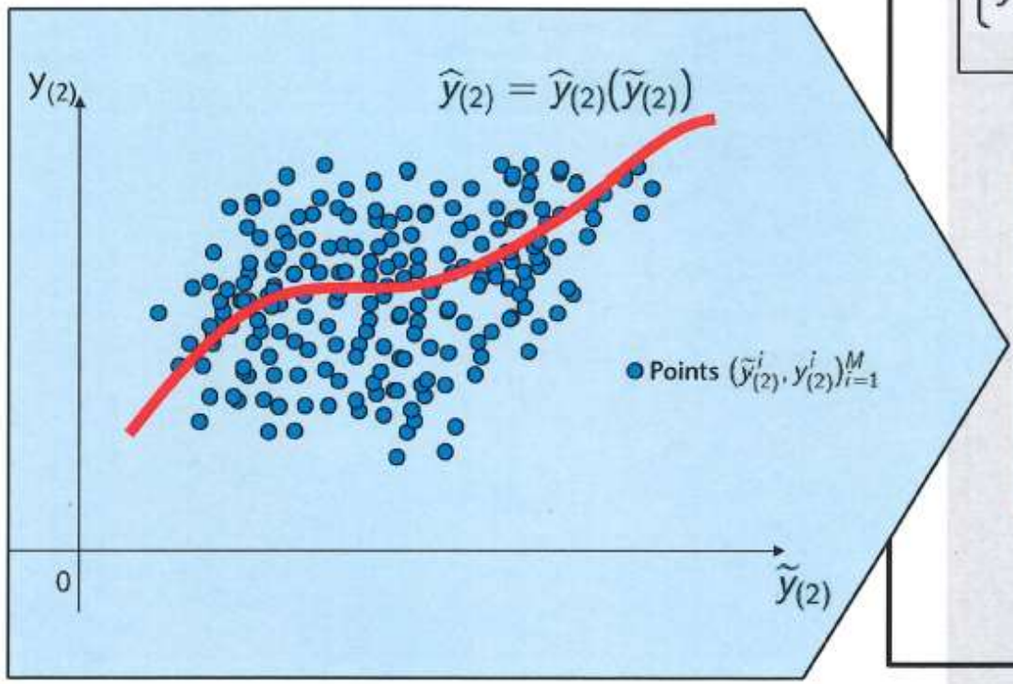
II Boosting stage

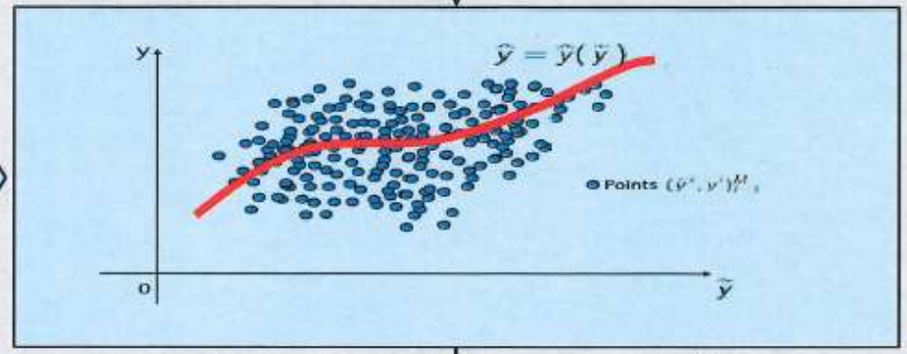
Do

$\{\vec{x}^i, y^i_{(2)}\}_{i=1}^N$ $\quad y^i_{(2)} = y^i_{(1)} - \widehat{y}_{(1)}(\widetilde{y}_{(1)}(\vec{x}^i))$

$\left\{\widetilde{y}_{(2)}(\vec{x}^i) = f_1^{(2)}(x_1^i) + f_2^{(2)}(x_2^i) + \ldots + f_d^{(2)}(x_d^i)\right\}_{i=1}^N$

1-dimensional regression





$\widehat{y}_{(2)}(\widetilde{y}_{(2)}(\vec{x}))$ – approximation of $y_{(2)}(\vec{x})$

While { <<stopping criteria>>}

Description of Kolmogorov-Pednault (KP) algorithm

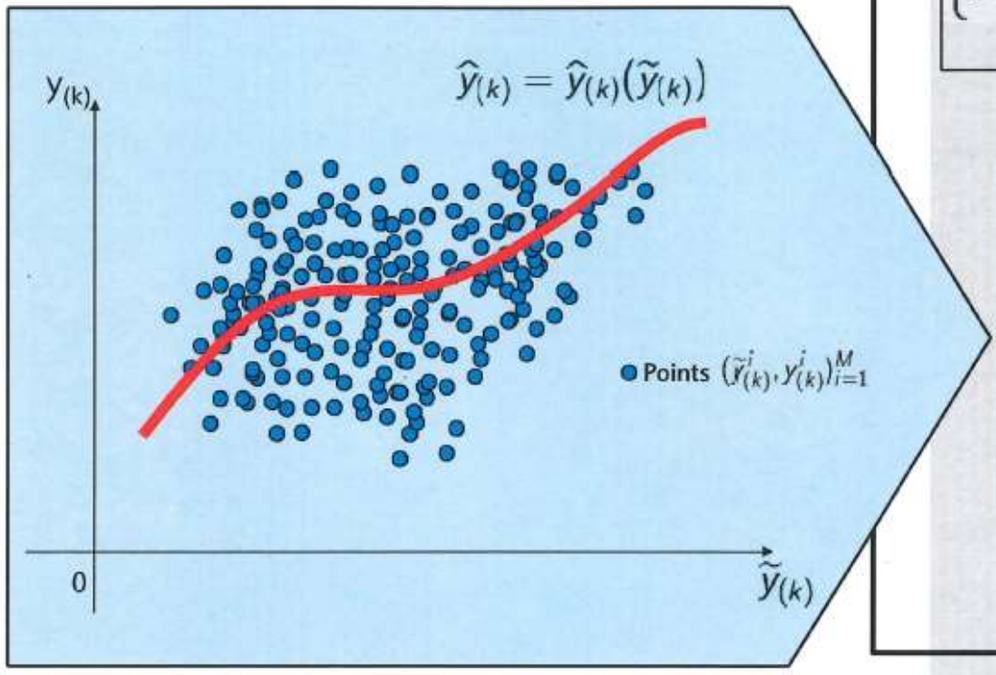
Kth Boosting stage

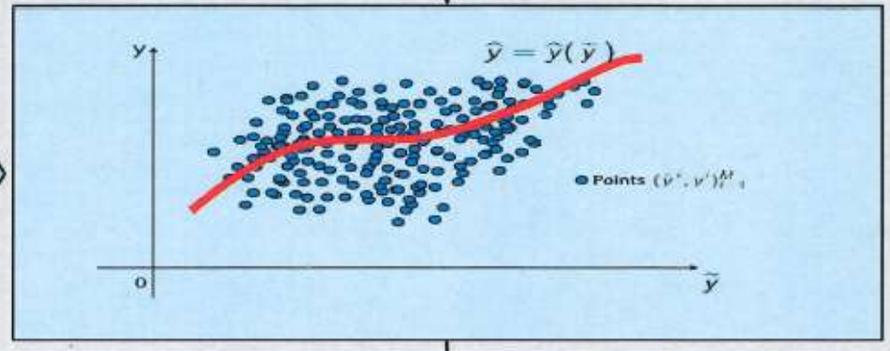
Do

$\{\vec{x}^i, y^i_{(k)}\}_{i=1}^N$ $y^i_{(k)} = y^i_{(1)} - \sum_{j=1}^{k-1} \widehat{y}_{(j)}(\widetilde{y}_{(j)}(\vec{x}^i))$

$\left\{\widetilde{y}_{(k)}(\vec{x}^i) = f_1^{(k)}(x_1^i) + f_2^{(k)}(x_2^i) + \ldots + f_d^{(k)}(x_d^i)\right\}_{i=1}^N$

1-dimensional regression





$\widehat{y}_{(k)}(\widetilde{y}_{(k)}(\vec{x}))$ – approximation of $y_{(k)}(\vec{x})$

While { <<stopping criteria>>}

Description of Kolmogorov-Pednault (KP) algorithm

What is common among the KP algorithm and the Kolmogorov's Superposition Theorem (KST) ?

- If **L** is the actual number of boosting stages in KP algorithm then the final approximation $\hat{y}(\vec{x})$ of $y(\vec{x})$ is equal to

$$\hat{y}(\vec{x}) = \hat{y}_{(1)}(\vec{x}) + \hat{y}_{(2)}(\vec{x}) + \ldots + \hat{y}_{(L)}(\vec{x})$$

- By construction, for each k = 1, … , L we have

$$\hat{y}_{(k)}(\vec{x}) = \hat{y}_{(k)}\left(f_1^{(k)}(x_1) + f_2^{(k)}(x_2) + \ldots + f_d^{(k)}(x_d) \right)$$

- The number of boosting stages L depends on the function $\hat{y}(\vec{x})$ and therefore is not fixed

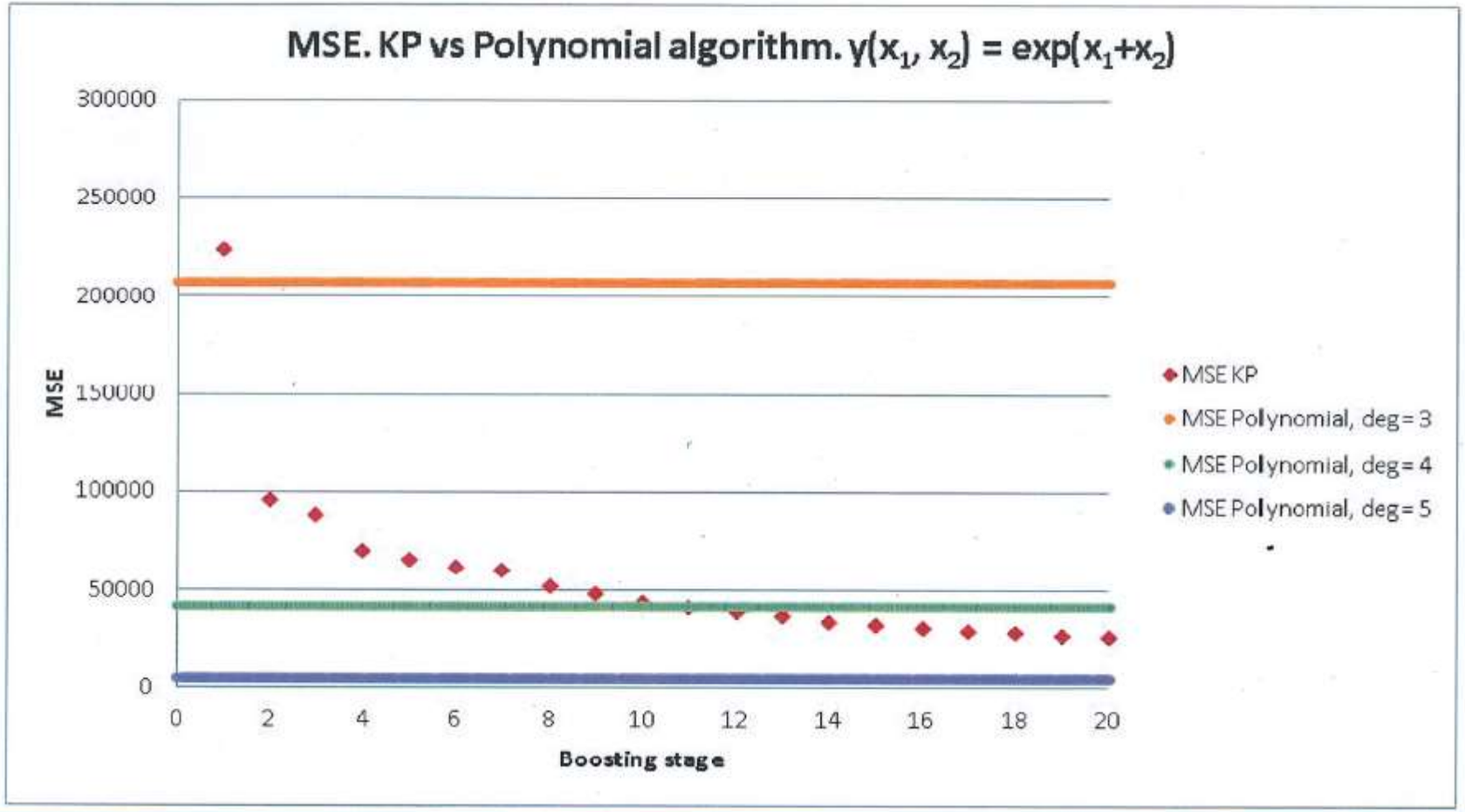
Comparison of KP and Polynomial algorithms

- Test function in 2D (d=2):

  $y(x_1, x_2) = \exp(x_1+x_2)$, $x_1, x_2 \in [0, 5]$;

$$MSE = \frac{1}{N-M}\sum_{i=M+1}^{N}(y^i - \hat{y}(\vec{x}^i))^2$$

- Input points: $y^i = \exp(x_1^i + x_2^i)$; x_1^i, x_2^i are uniformly distributed random points on [0, 5], i = 1, 2, …, N; N = 15000, M = 10000



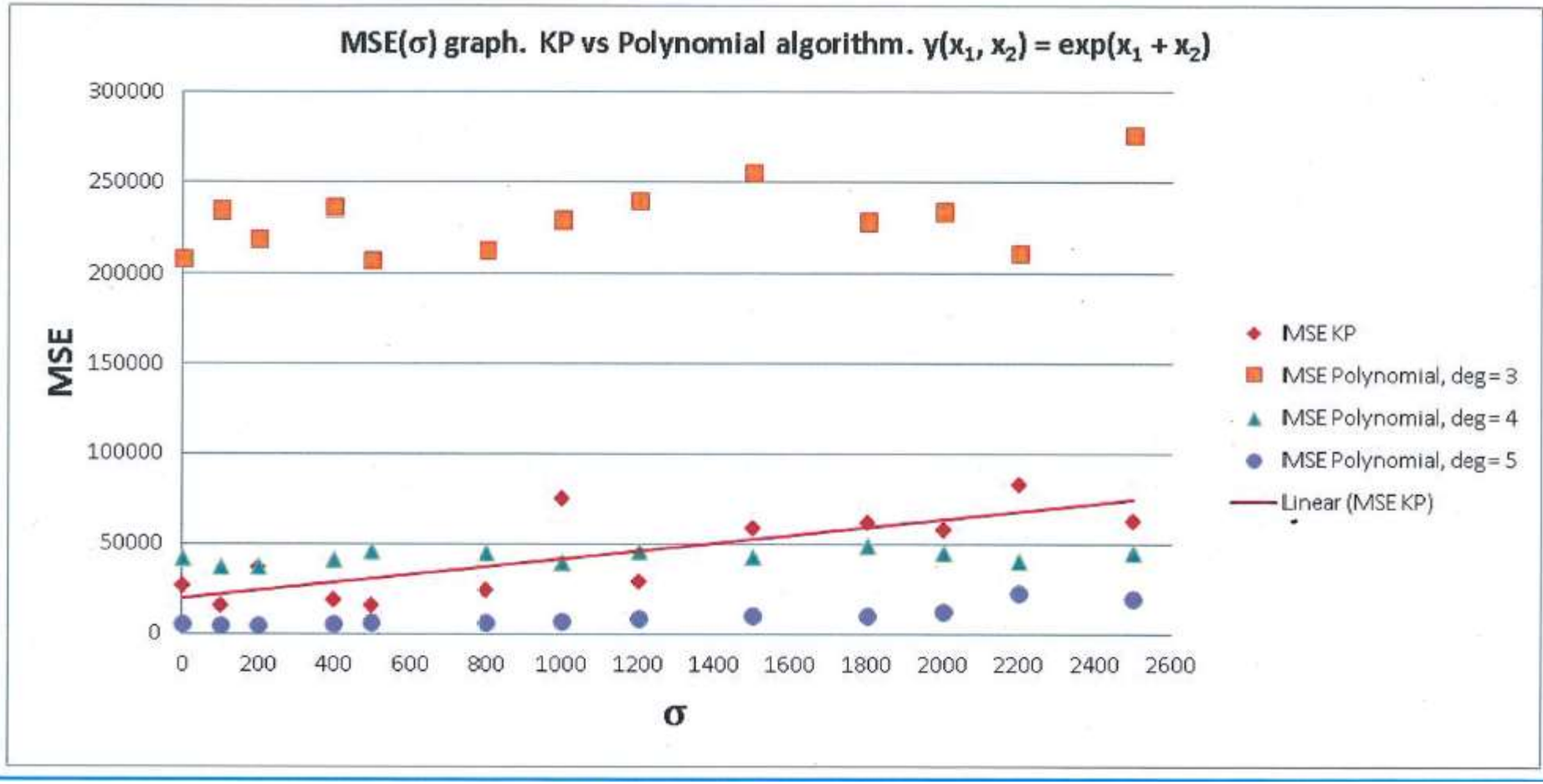
Comparison of KP and Polynomial algorithms

- Test function in 2D (d=2):

  $y(x_1, x_2) = \exp(x_1+x_2)$, $x_1, x_2 \in [0, 5]$;

$$MSE = \frac{1}{N-M}\sum_{i=M+1}^{N}(y(\vec{x}^i) - \hat{y}(\vec{x}^i))^2$$

- Input points: $y^i = \exp(x_1^i + x_2^i) + \varepsilon^i$; x_1^i, x_2^i are uniformly distributed random points on [0,5], $\varepsilon^i \sim \mathcal{N}(0, \sigma^2)$, i = 1, 2, …, N; N = 15000, M = 10000



Comparison of KP and Polynomial algorithms

- Test function in 2D (d=2):

$$y(x_1, x_2) = \begin{cases} 0.3, & x_1, x_2 \in [0; 0.5) \\ 1, & x_1 \in [0; 0.5),\ x_2 \in [0.5; 1] \\ 0, & x_1 \in [0.5; 1],\ x_2 \in [0; 0.5) \\ 0.6, & x_1, x_2 \in [0.5; 1] \end{cases}$$

- Input points: $y^i = y(x_1^i, x_2^i)$;

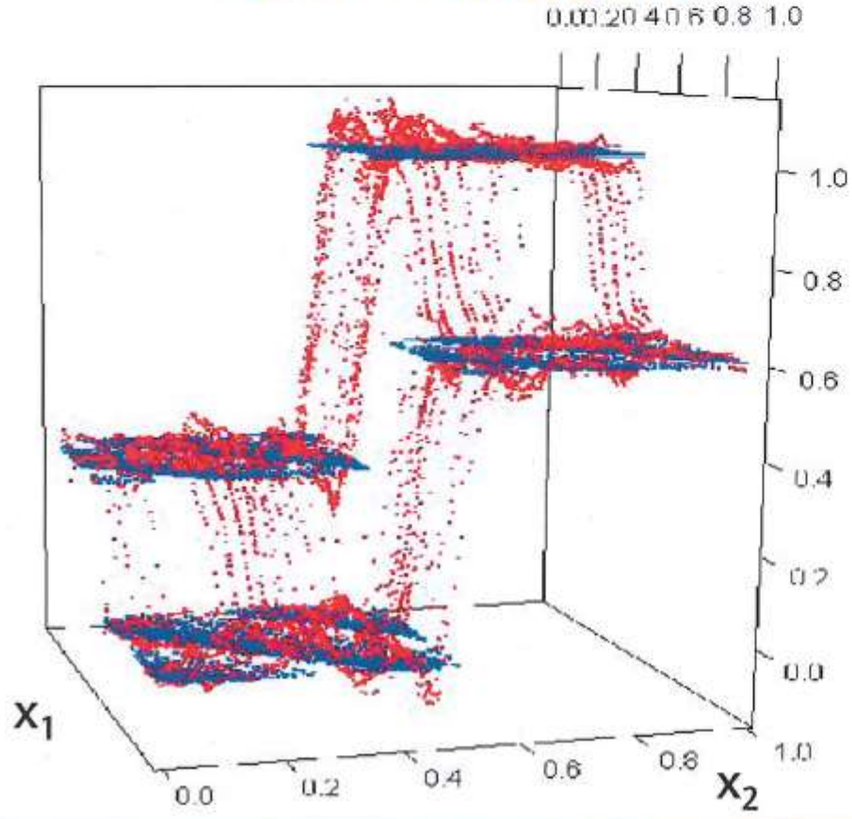
  $x_1^i, x_2^i \sim U[0, 1]$, $i = 1, 2, \ldots, N$;

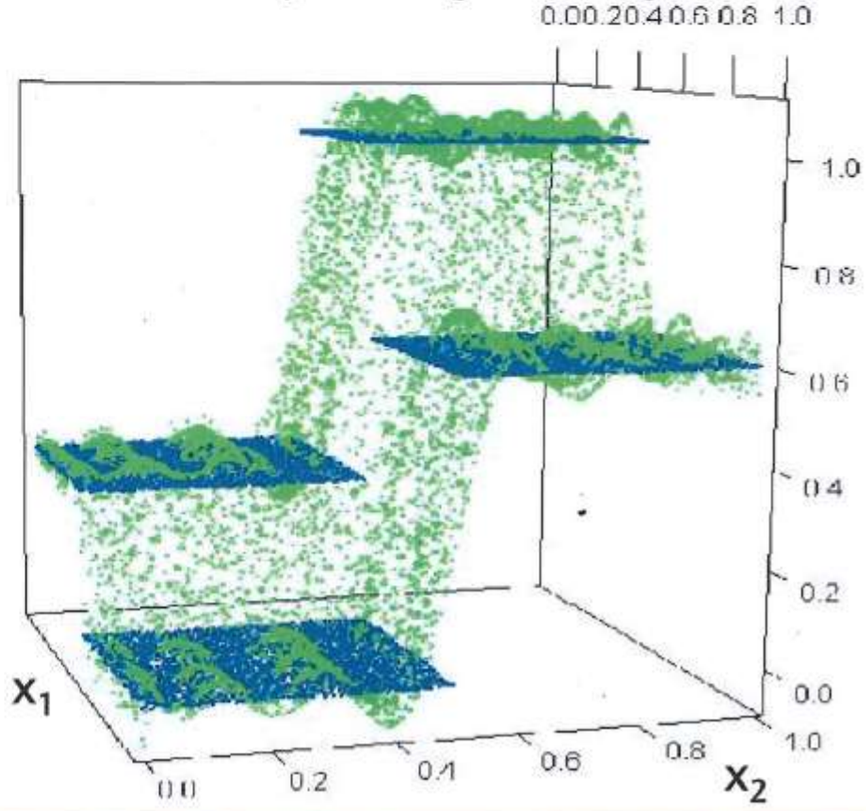
  $N = 15000$, $M = 10000$

$y(x_1, x_2)$

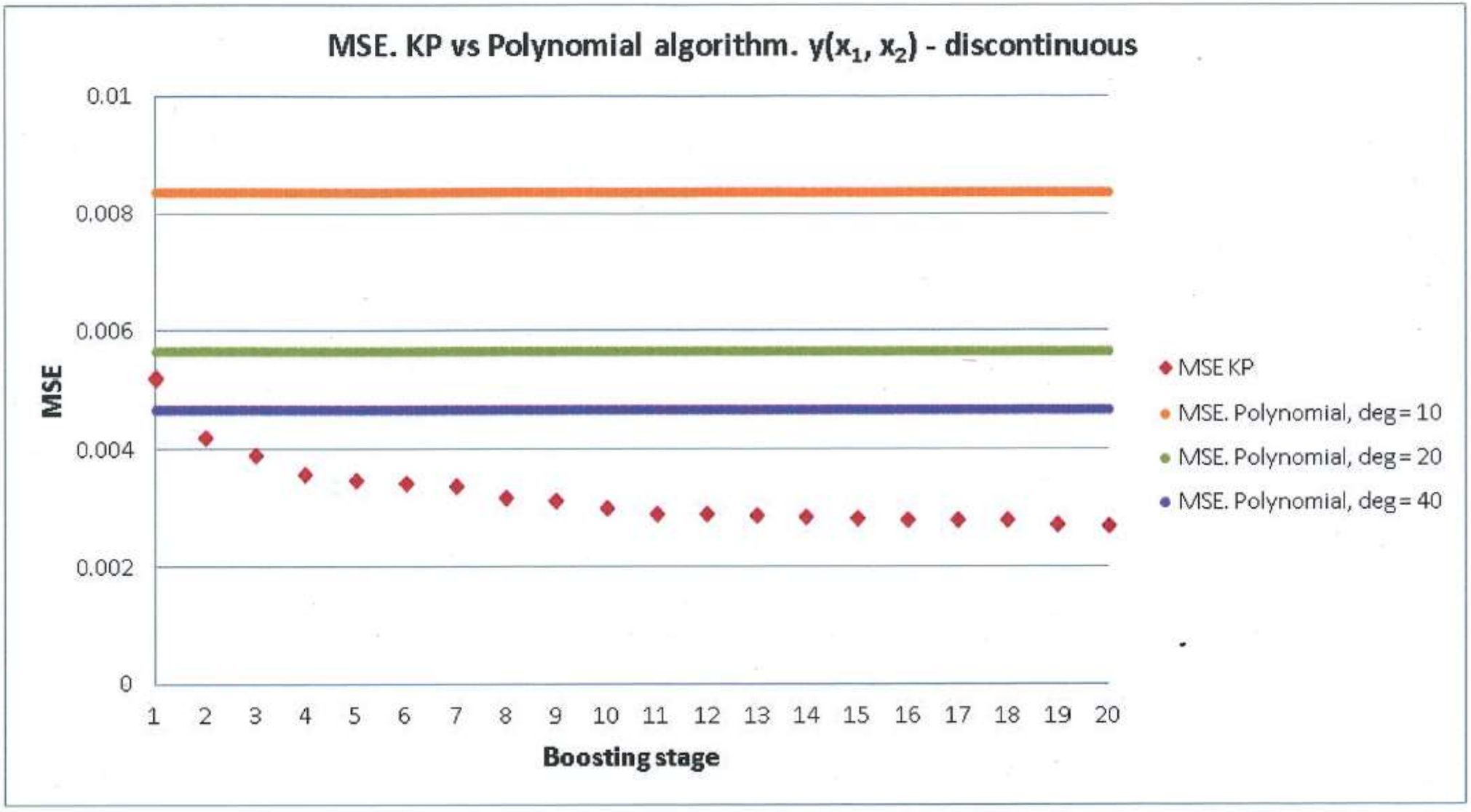
KP algorithm

Polynomial algorithm, deg = 20





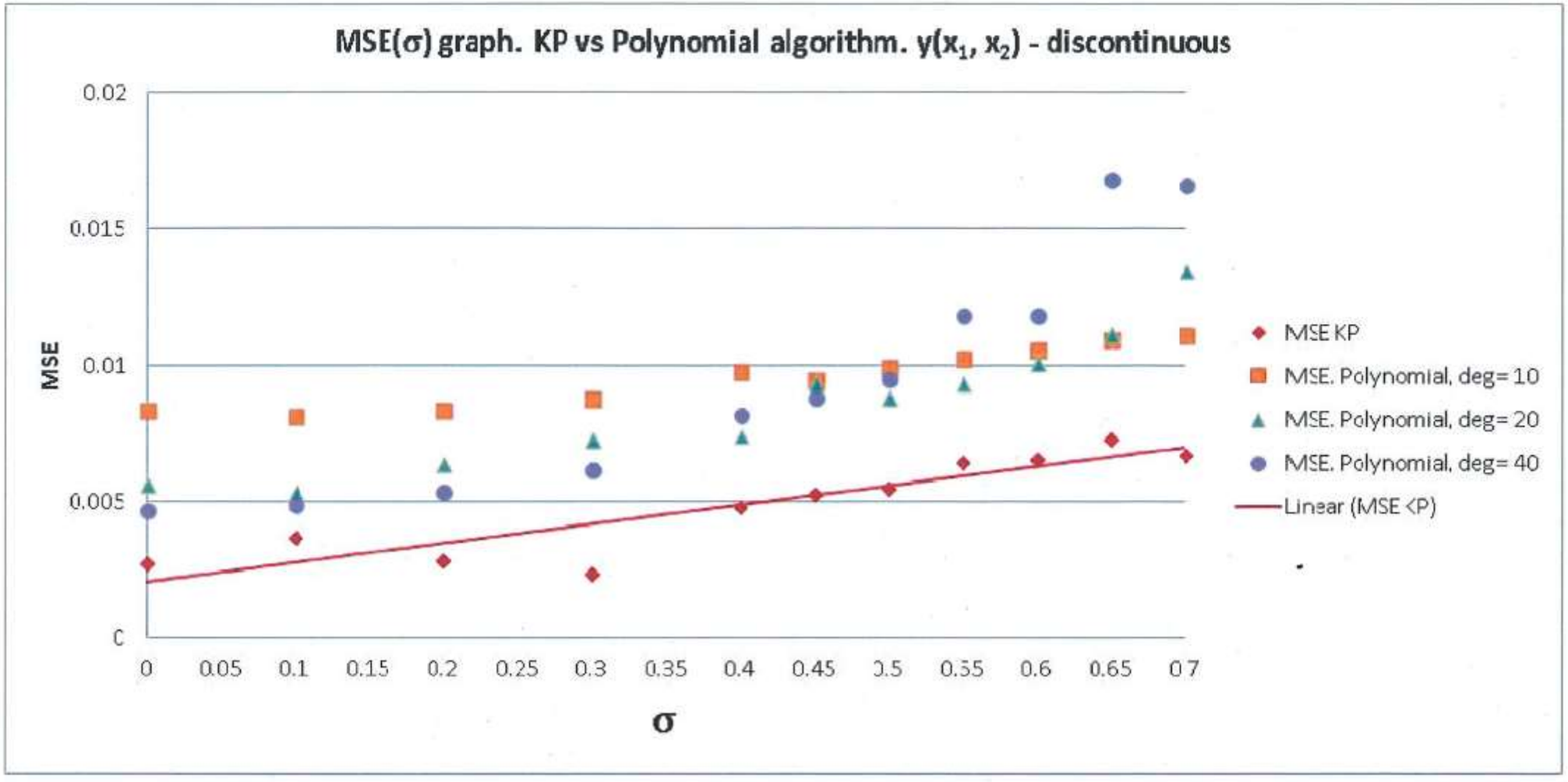
Comparison of KP and Polynomial algorithms



Comparison of KP and Polynomial algorithms

Input points: $y^i = y(x_1^i, x_2^i) + \varepsilon^i$; $x_1^i, x_2^i \sim U[0, 1]$, $\varepsilon^i \sim \mathcal{N}(0, \sigma^2)$, $i = 1, 2, \ldots, N$;

$N = 15000$, $M = 10000$



TARN instrument (FX)

Example in 2D (d=2) : TARN trade

- Let FX_1 = AUDJPY, FX_2 = USDJPY, spread = $FX_1 - FX_2$
- Let $T_1 < T_2 < \ldots < T_n$ be n **fixing dates**, $T > T_n$ – **valuation date**
- $A(T_i) = \begin{cases} 1 & \text{if } \text{spread}(T_i) > B \\ 0 & \text{if } \text{spread}(T_i) \le B \end{cases}$ – accrued amount at fixing date T_i, B – **spread boundary**

$$A(t) = \sum_{i:\, T_i \le t} A(T_i) \quad \text{– accrued amount until date t}$$

- **Payoff** (T) = $\sum_{i=1}^{n} \text{Notinal}_i \cdot (FX_1(T_i) - FX_2(T_i) - K)$. *We have a **Basket Forward Strip***

which pays coupons over time until the accrued amount A(·) reaches a given TARN level

x_1 = accrued amount,
x_2 = spread = $FX_1 - FX_2$

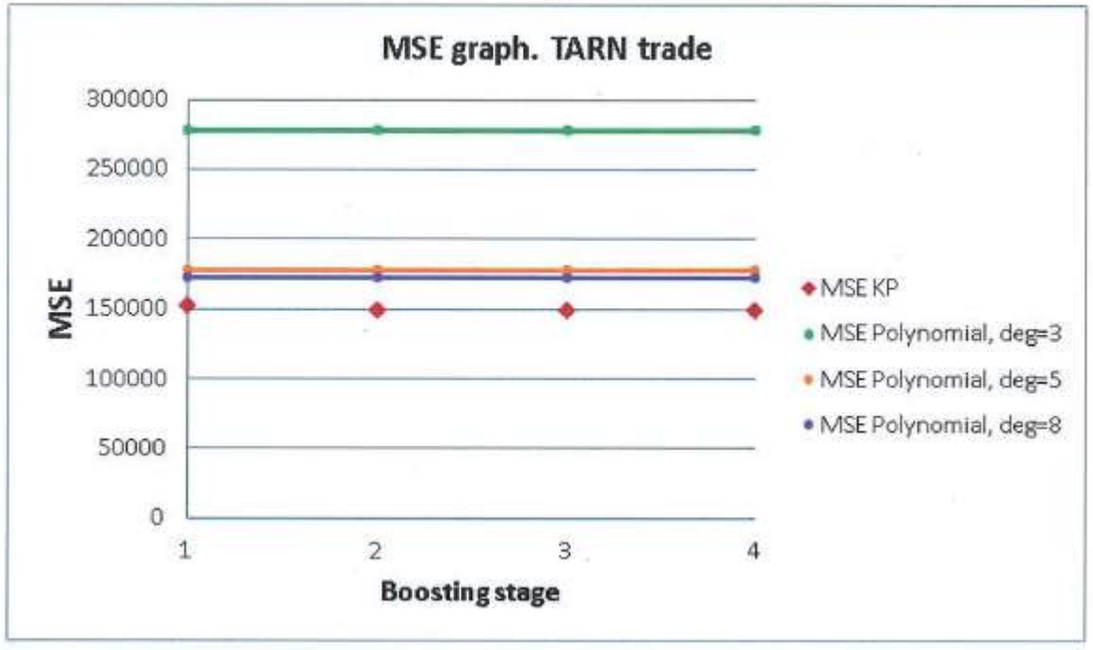
y – value of the trade close to expiry

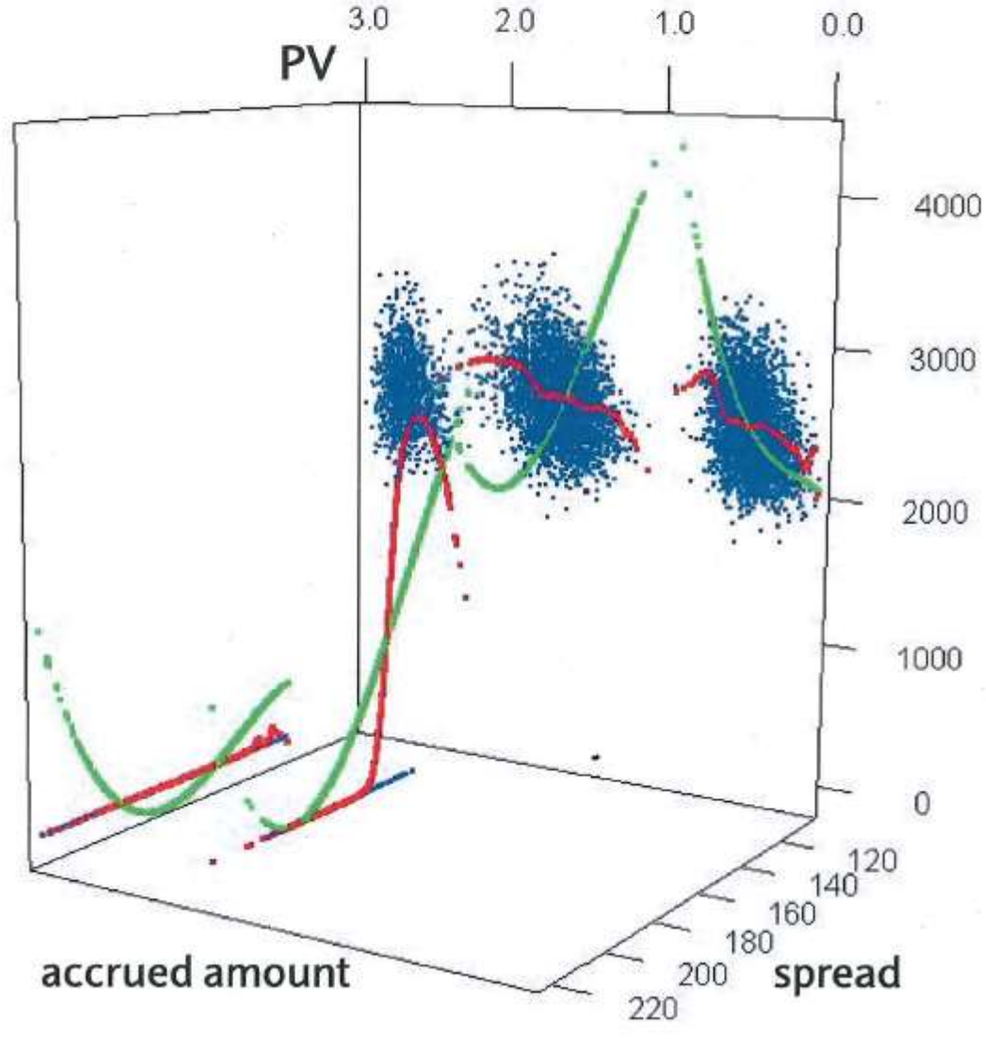
Comparison of KP and Polynomial algorithms

Example in 2D (d=2) : **TARN trade** TARN Level = 3

x_1 = accrued amount, x_2 = spread = $FX_1 - FX_2$ $(x_1^i, x_2^i, y^i)_{i=1}^N$, N = 15000 are input points

y – value of the trade close to expiry





Conclusions

Summary of work

- A regression method has been developed based on KST
  - ➢ algorithm provide good approximation for discontinuous functions; approximation power for continuous and smooth functions is acceptable
  - ➢ no additional input parameters (like polynomial degree) are required

Future work

- Make a performance optimization of the algorithms
- Try different 1D smoothers (various kernel smoothers, cubic smoothing splines, regression trees). Make a comparative analysis
- Analyze the internal parameters sensitivity of developed algorithms

References

[1] **Kolmogorov, A. N.** (1957) *On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition,* Dokl. Akad. Nauk USSR, vol. 114, pp. 953-956

[2] **Pednault, E.** (2006) *Transform regression and Kolmogorov Superposition Theorem,* Proceedings of the 2006 SIAM International Conference on Data Mining, pp. 35-46

[3] **Sprecher, D.** (1996) *A numerical implementation of Kolmogorov's superpositions*, Neural Networks, vol. 9, No. 5, pp. 765-772

[4] **Brattka, V.** (2000) *A computable version of Kolmogorov Superposition Theorem,* Computability and Complexity in Analysis, vol. 272, pp. 7-22

[5] **Hastie, T., Tibshirani, R., Friedman, J.** (2009) *The Elements of Statistical Learning,* Springer

Thank you for your attention!